```
analogWrite(ledPin, EEPROM.read(i));
```

This is the last line of code that we changed. Instead of writing the value of the `flicker[]` array to the analog pin, we instead read from each of the EEPROM addresses and write that value. The syntax for the `EEPROM.read()` is pretty simple, as follows:

```
EEPROM.read(address)
```

All we need to do is specify the EEPROM address that we want to read the data from, a number from 0–1023, and that value will be returned back. And that concludes the quick rundown on reading and writing data to our microcontroller's EEPROM. It's probably not the best example because our constant reading of the brightness values will potentially use up our read/write lifetime in about 10 hours, causing our EEPROM to stop working, so I don't recommend leaving the sketch running for too long.

Instead, the EEPROM is a good place to store sensor readings every minute or so over a few hours for a low-cost data logger. Because the memory is non-volatile and is not used by the Arduino otherwise, this data will remain in memory regardless of power or additional code being loaded in program memory. Only until a running program overwrites the data, the EEPROM data will persistently stick around. This is also a good reason for storing certain configuration settings that might need to get dialed in while the code is running, but if the power is reset or the program is reloaded we would still want those settings to remain. The BlinkM from the last chapter uses its EEPROM to store similar configuration settings.

Summary

If you're still hanging on after some of that less-than- friendly code, then good for you. Anytime I have to dredge through any of that fringe stuff that hasn't already been brought into the Arduino fold, I am plenty thankful for everything that has already been wrapped up for us. Arrays are a pretty useful tool in our Arduino projects and we could use two chapters to get through everything that can be done with them, but we still have more things to talk about. Arrays work to make all manner of lists of things from pin numbers, to output levels, sensor readings, and even text messages. They can be rolled into loops and functions with amazing ease because of their indexed data elements. We just need to be careful how we handle our memory usage when using them and stay on the lookout for strange behavior that might be caused by limited RAM.

Next up, we are going to look at a few more libraries written for particular hardware and devices that add functionality to the Arduino platform. We'll begin by revisiting the `LiquidCrystal` library before moving on to a couple of other libraries for different types of motors before ending the chapter with finding another way to store lots of data on a convenient memory card. What's important here is that we are now going to get back to making things move again, so the next chapter should be a lot of fun!